

# Compiler Design

## An Introduction

Samit Biswas<sup>1</sup>

<sup>1</sup>Department of Computer Science and Technology,  
Indian Institute of Engineering Science and Technology, Shibpur  
Email: [samit@cs.iiests.ac.in](mailto:samit@cs.iiests.ac.in)

# Table of Contents

- 1 Pre-requisite courses
- 2 Introduction To Compiler Design
- 3 The Phases of a Compiler
- 4 Compiler Construction Tools

# Pre-requisite courses

- Strong **programming** background in C, C++ or Java

# Pre-requisite courses

- Strong **programming** background in C, C++ or Java
- Formal Language and Automata Theory (REs, NFAs, DFAs, CFG, PDA).

# Pre-requisite courses

- Strong **programming** background in C, C++ or Java
- Formal Language and Automata Theory (REs, NFAs, DFAs, CFG, PDA).
- Assembly Language Programming and Machine Architecture.

# Table of Contents

- 1 Pre-requisite courses
- 2 Introduction To Compiler Design**
- 3 The Phases of a Compiler
- 4 Compiler Construction Tools

*Understanding compilers is useful*

- **Why** study compilers?

*Understanding compilers is useful*

- **Why study compilers?**
  - Compilers are **complex** programs.

*Understanding compilers is useful*

- **Why study compilers?**

- Compilers are **complex** programs.
- You often need compilers  
(whenever you write and run a program)

*Understanding compilers is useful*

- **Why study compilers?**

- Compilers are **complex** programs.
- You often need compilers  
(whenever you write and run a program)
- If you understand **how compilers work**,  
you'll know what to expect  
and how to get the best out of them

*Building compilers is useful.*

- **Compiler-like** programs are everywhere.

*Building compilers is useful.*

- **Compiler-like programs are everywhere.**

All sorts of programs can be viewed as compilers. For example,

query languages (GraphQL)

text processors (sed, grep, ...)

build systems (make)

game engine scripting (Lua)

financial contracts (MLFI)

music systems (Csound)

text editors (emacs)

VHDL

R

browser engines (WebKit)

document processors (LATEX)

GitHub Actions

blockchain platforms (Solidity)

legal contracts (Catala)

interactive testing systems (expect)

compiler compilers (yacc)

wikis (MediaWiki)

- **What is a compiler?**

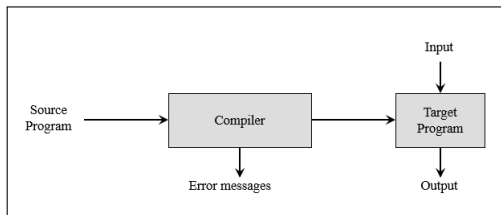
- **What is a compiler?**

- Before a program can be run, it first must be translated into a form (**Target Program**), which can be executed by a computer.

# Introduction To Compiler Design

## • What is a compiler?

- Before a program can be run, it first must be translated into a form (**Target Program**), which can be executed by a computer.
- **Translation** of a program written in a source language into a semantically equivalent program written in a target language



# Introduction To Compiler Design

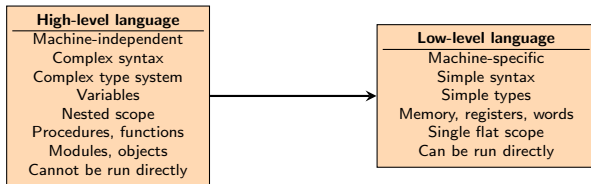
- A compiler is a program that translates (or compiles) a program written in a **high-level** programming language (the source language) that is suitable for human programmers into the **low-level** machine language (target language) that is required by computers.

# Introduction To Compiler Design

- A compiler is a program that translates (or compiles) a program written in a **high-level** programming language (the source language) that is suitable for human programmers into the **low-level** machine language (target language) that is required by computers.
- During this process, the compiler also tries to identify and report obvious programming errors that are detected during translation.

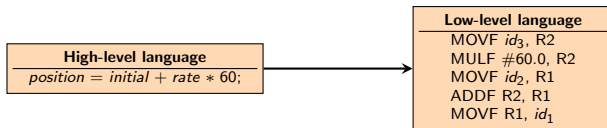
# Introduction To Compiler Design

- A compiler is a program that translates (or compiles) a program written in a **high-level** programming language (the source language) that is suitable for human programmers into the **low-level** machine language (target language) that is required by computers.
- During this process, the compiler also tries to identify and report obvious programming errors that are detected during translation.



# Introduction To Compiler Design

- Example:  $C \longrightarrow \text{Assembly code}$





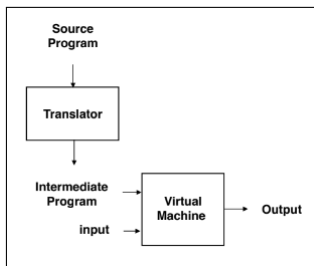
# Compiler vs. Interpreter

- **Compiler vs. Interpreter**

| Compiler                                                        | Interpreter                                             |
|-----------------------------------------------------------------|---------------------------------------------------------|
| Takes the entire program as input                               | Take single instruction as input                        |
| It is Faster                                                    | It is Slower                                            |
| Intermediate object code is generated.                          | no intermediate code is generated;                      |
| Errors are displayed after the entire program has been checked. | Errors are displayed for every instruction interpreted. |
| Ex: C, C++.                                                     | Ex: python, Ruby, basic.                                |

- **Bytecode & Virtual Machine**

A Java source program may first be compiled into an intermediate form called bytecodes. The bytecodes are then interpreted by a virtual machine. A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network.



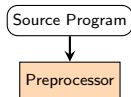
# A typical Language Processing System

## A typical Language Processing System

Source Program

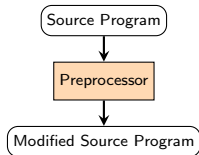
# A typical Language Processing System

## A typical Language Processing System



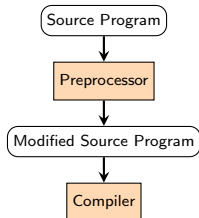
# A typical Language Processing System

## A typical Language Processing System



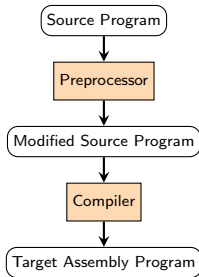
# A typical Language Processing System

## A typical Language Processing System



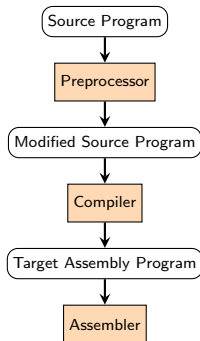
# A typical Language Processing System

## A typical Language Processing System



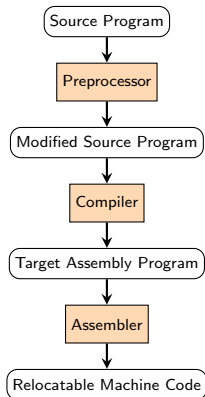
# A typical Language Processing System

## A typical Language Processing System



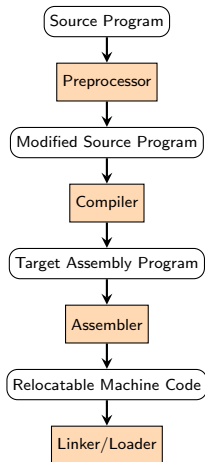
# A typical Language Processing System

## A typical Language Processing System



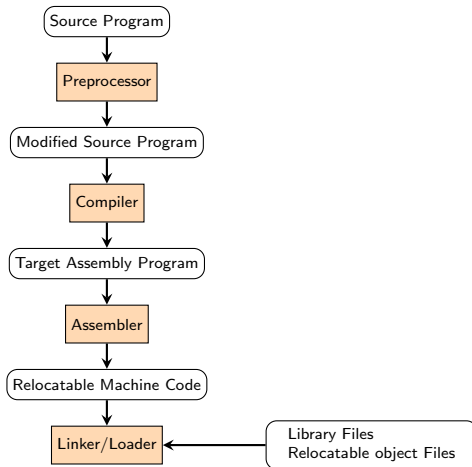
# A typical Language Processing System

## A typical Language Processing System



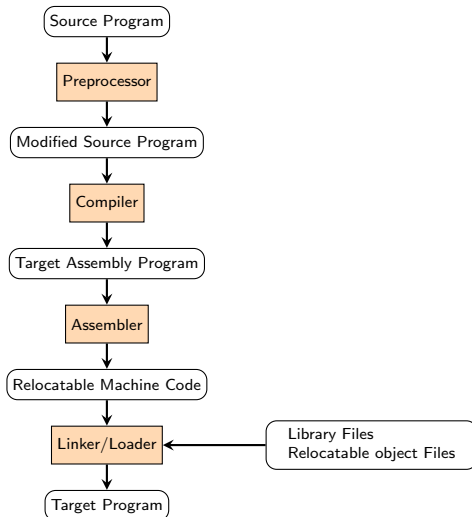
# A typical Language Processing System

## A typical Language Processing System



# A typical Language Processing System

## A typical Language Processing System



## Issues related to Compiler Design:

- Correctness

## Issues related to Compiler Design:

- Correctness
- Speed (runtime and compile time)
  - Degrees of optimization
  - Multiple passes

## Issues related to Compiler Design:

- Correctness
- Speed (runtime and compile time)
  - Degrees of optimization
  - Multiple passes
- Space

## Issues related to Compiler Design:

- Correctness
- Speed (runtime and compile time)
  - Degrees of optimization
  - Multiple passes
- Space
- Feedback to user

## Issues related to Compiler Design:

- Correctness
- Speed (runtime and compile time)
  - Degrees of optimization
  - Multiple passes
- Space
- Feedback to user
- Debugging

# Table of Contents

- 1 Pre-requisite courses
- 2 Introduction To Compiler Design
- 3 The Phases of a Compiler**
- 4 Compiler Construction Tools

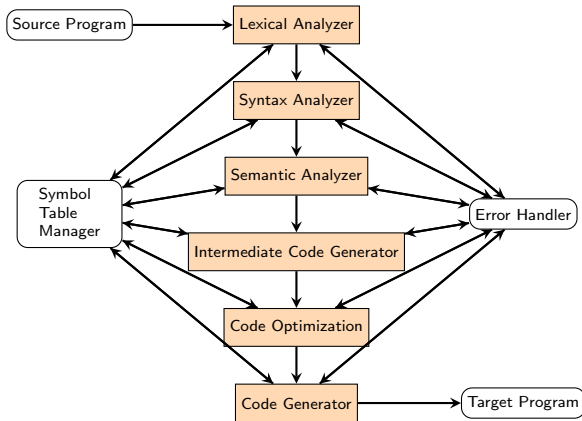
# Analysis - Synthesis Model of Compilation

## There are two parts of compilation:-

- **Analysis** - It breaks up the source program into constituent pieces and creates an intermediate representation of the source program. If the analysis part detects that the source program is either syntactically ill formed or semantically unsound, then it must provide informative messages, so the user can take corrective action.
- **Synthesis** - It constructs the desired target program from the intermediate representation and the information in the symbol table.

# The Phases of a Compiler

- Conceptually, a compiler operates in phases, each of which translates the source program from one representation to another.



# Translation of an Assignment Statement

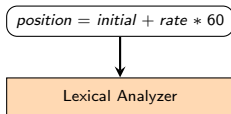
## Translation of an Assignment Statement:

- Lexical Analyser takes the source program as input and produces a long string of tokens.
- Syntax Analyser takes an out of lexical analyser and produce a large tree.
- Semantic Analyser takes an output of Syntax analyser and produces another tree.
- Similarly Intermediate code generator takes a tree as an input produced by Semantic analyser and produces Intermediate code.

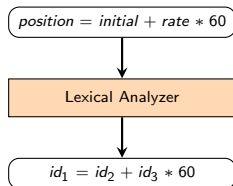
# The Phases of a Compiler

$$position = initial + rate * 60$$

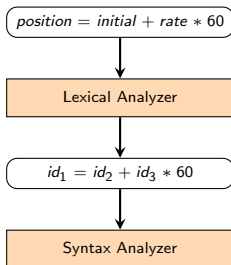
# The Phases of a Compiler



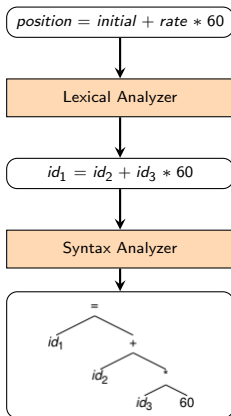
# The Phases of a Compiler



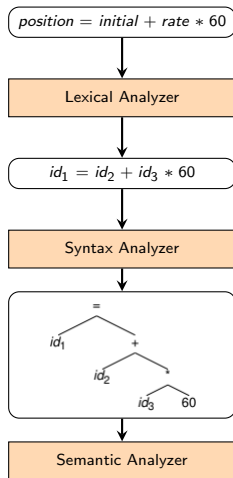
# The Phases of a Compiler



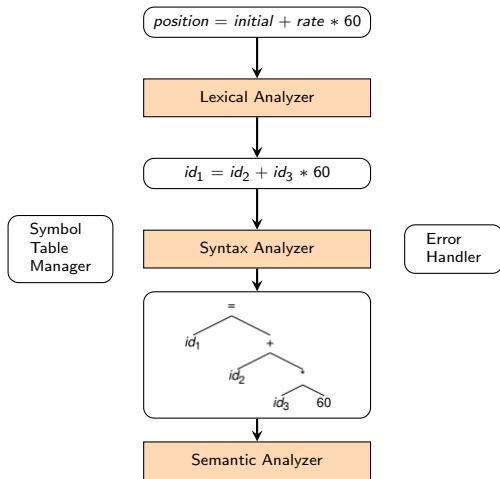
# The Phases of a Compiler



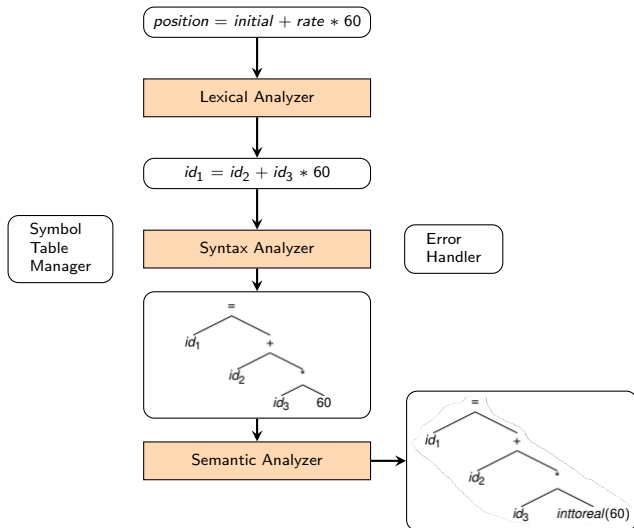
# The Phases of a Compiler



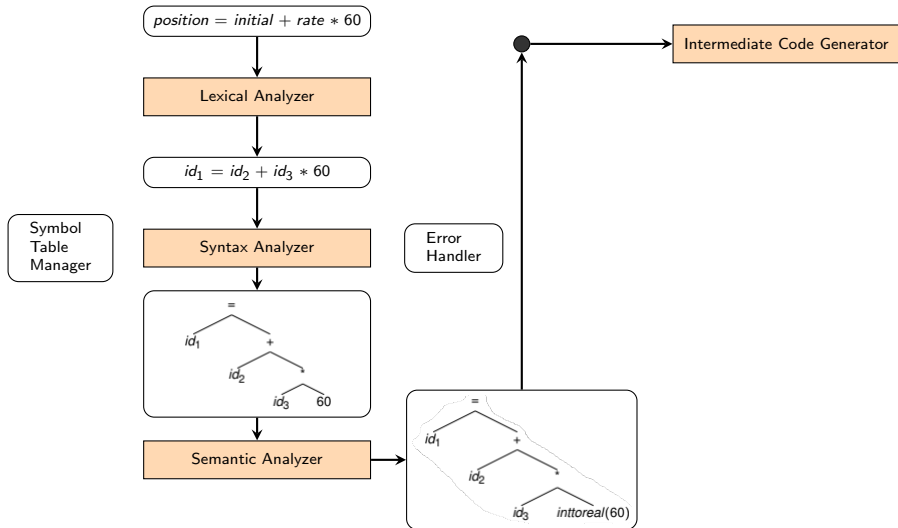
# The Phases of a Compiler



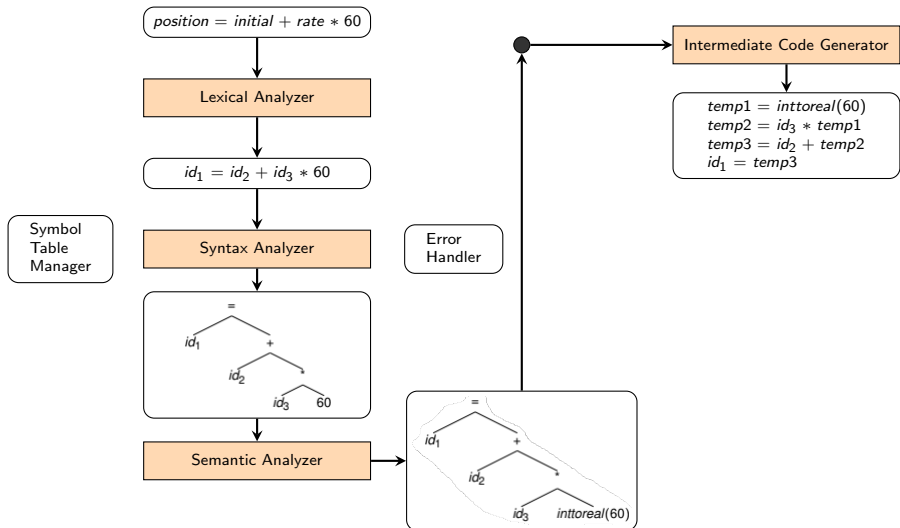
# The Phases of a Compiler



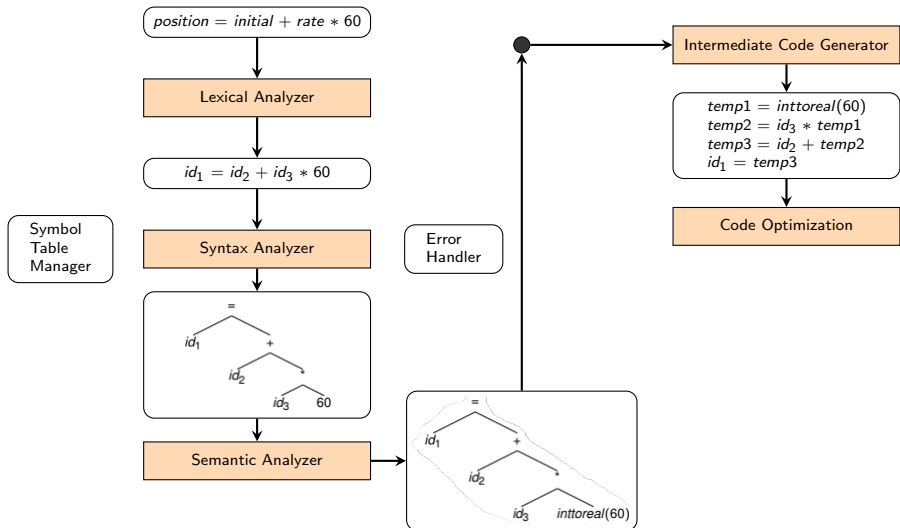
# The Phases of a Compiler



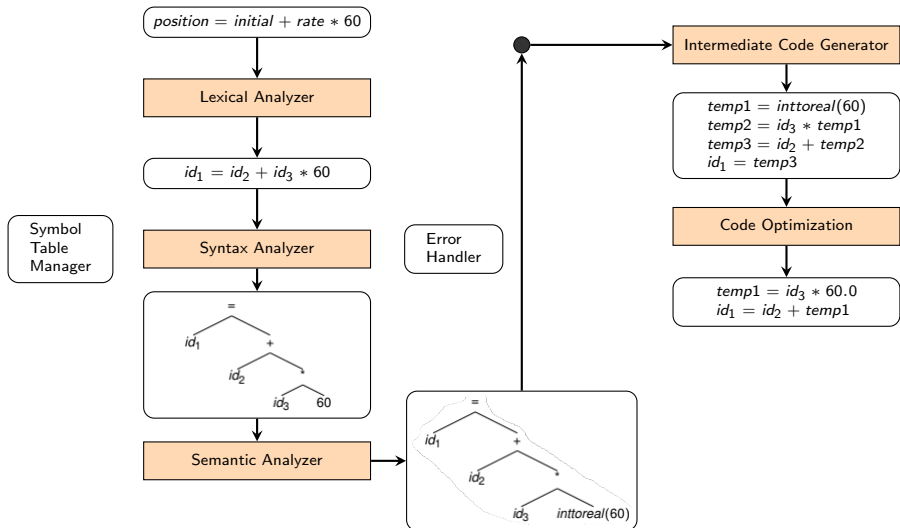
# The Phases of a Compiler



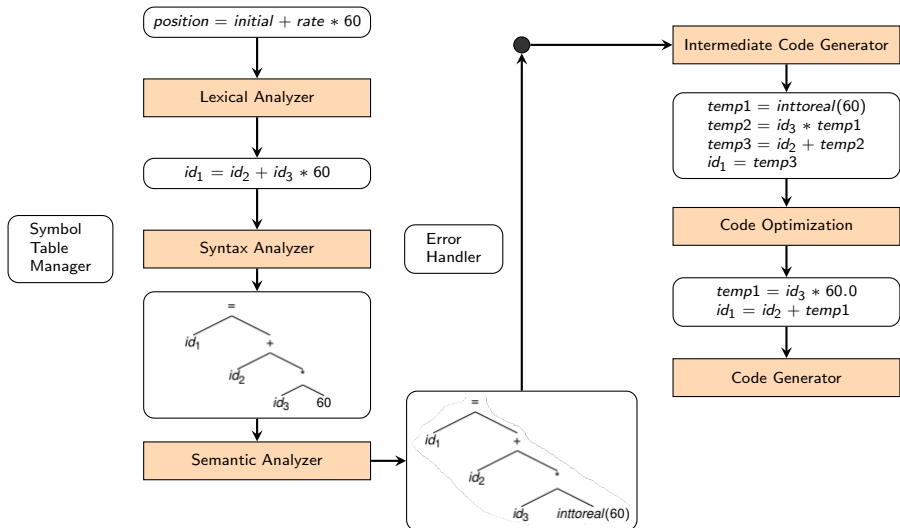
# The Phases of a Compiler



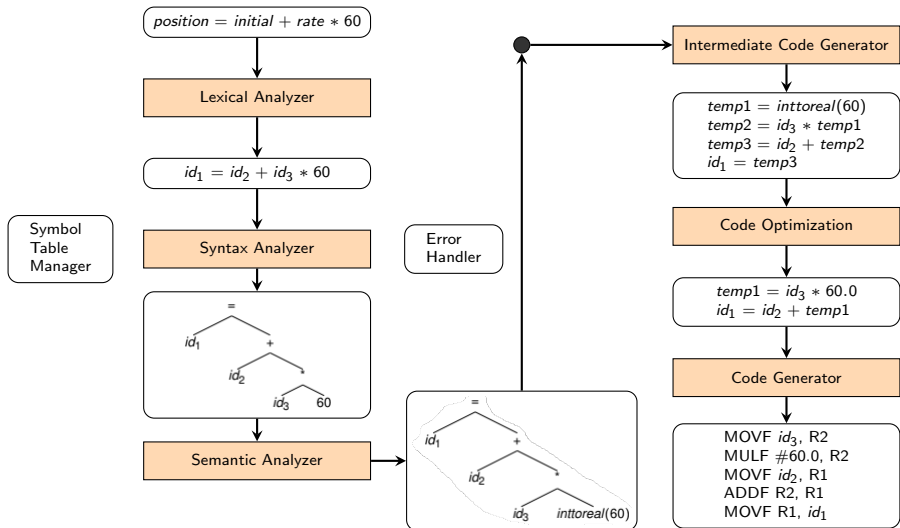
# The Phases of a Compiler



# The Phases of a Compiler



# The Phases of a Compiler



# Table of Contents

- 1 Pre-requisite courses
- 2 Introduction To Compiler Design
- 3 The Phases of a Compiler
- 4 **Compiler Construction Tools**

**Software Tools are available to implement compiler phases-**

## Software Tools are available to implement compiler phases-

- **Scanner Generators** that produce lexical analysers from a regular expression.

## Software Tools are available to implement compiler phases-

- **Scanner Generators** that produce lexical analysers from a regular expression.
- **Parser Generators** that automatically produce syntax analyser.

## Software Tools are available to implement compiler phases-

- **Scanner Generators** that produce lexical analysers from a regular expression.
- **Parser Generators** that automatically produce syntax analyser.
- Syntax Directed translation Engine.

## Software Tools are available to implement compiler phases-

- **Scanner Generators** that produce lexical analysers from a regular expression.
- **Parser Generators** that automatically produce syntax analyser.
- Syntax Directed translation Engine.
- **Data-flow analysis** engines that facilitate the gathering of information about how values are transmitted from one part of a program to each other part. Data-flow analysis is a key part of code optimization.

## Software Tools are available to implement compiler phases-

- **Scanner Generators** that produce lexical analysers from a regular expression.
- **Parser Generators** that automatically produce syntax analyser.
- Syntax Directed translation Engine.
- **Data-flow analysis** engines that facilitate the gathering of information about how values are transmitted from one part of a program to each other part. Data-flow analysis is a key part of code optimization.
- **Code Generator** that produce a code generator from a collection of rules for translating each operation of the intermediate language into the machine language for a target machine.

## Software Tools are available to implement compiler phases-

- **Scanner Generators** that produce lexical analysers from a regular expression.
- **Parser Generators** that automatically produce syntax analyser.
- Syntax Directed translation Engine.
- **Data-flow analysis** engines that facilitate the gathering of information about how values are transmitted from one part of a program to each other part. Data-flow analysis is a key part of code optimization.
- **Code Generator** that produce a code generator from a collection of rules for translating each operation of the intermediate language into the machine language for a target machine.

# The Evolution of Programming Language

## Classification by generation

- **First-generation languages:** machine languages.
- **Second-generation:** assembly languages
- **Third-generation:** higher-level languages like Fortran, Cobol, Lisp, C, C++, C#, and Java.
- **Fourth-generation languages:** languages designed for specific applications like NOMAD for report generation, SQL for database queries, and Postscript for text formatting.
- **Fifth-generation language** has been applied to logic- and constraint-based languages like Prolog and OPS5.

# Impacts on Compilers

- The advances in programming languages placed new demands on compiler writers.
- Compiler writers would take maximal advantage of the new hardware capabilities.
- Good software-engineering techniques are essential for creating and evolving modern language processors.

- Techniques used in a lexical analyzer can be used in **text editors**, **information retrieval system**, and **pattern recognition** programs.

# Other Applications

- Techniques used in a lexical analyzer can be used in **text editors**, **information retrieval system**, and **pattern recognition** programs.
- Techniques used in a parser can be used in a query processing system such as SQL.

# Other Applications

- Techniques used in a lexical analyzer can be used in **text editors**, **information retrieval system**, and **pattern recognition** programs.
- Techniques used in a parser can be used in a query processing system such as SQL.
- Many software having a complex front-end may need techniques used in compiler design. (A symbolic equation solver which takes an equation as input. That program should parse the given input equation)

- Techniques used in a lexical analyzer can be used in **text editors**, **information retrieval system**, and **pattern recognition** programs.
- Techniques used in a parser can be used in a query processing system such as SQL.
- Many software having a complex front-end may need techniques used in compiler design. (A symbolic equation solver which takes an equation as input. That program should parse the given input equation)
- Most of the techniques used in compiler design can be used in **Natural Language Processing (NLP)** systems

- Alfred V. Aho, Ravi Sethi, Jeffrey D Ullman, “Compilers Principles Techniques and Tools”, Pearson Education.